

打造第一個 AI 幫手：新手研討會

來源：<https://codelabs.developers.google.com/companion-adk-beginner/instructions?hl=zh-tw>

步驟數：6

在本程式碼研究室中，您將使用 Python ADK 和 Gemini CLI，建構智慧型對話式 AI 幫手。您將為代理程式設定自訂個性、網路搜尋功能，以及獨一無二的 AI 生成虛擬替身。

目錄

1. [課程內容](#)
2. [事前準備](#)
3. [使用 Gemini CLI 建立角色](#)
4. [強化代理程式的 Persona](#)
5. [新增近期事件的基礎](#)
6. [自訂隨播廣告的外觀 \(選用\)](#)

步驟 1 · 約 0 分鐘

1. 課程內容

- 如何使用 Gemini CLI 生成完整的 ADK 代理程式設定。
- 如何改善指令，提升代理的個性。
- 如何為代理程式「建立基準」，讓代理程式能夠根據 `google_search` 回答近期活動相關問題。
- 如何使用 MCP 伺服器 and Imagen，為夥伴模式生成自訂虛擬化身。

AI Companion 應用程式

在本程式碼研究室中，您將打造生動活潑的互動式 AI 夥伴。這不只是標準的文字輸入/輸出聊天機器人，假設網頁上住著一個角色，你輸入訊息後，角色會看著你並大聲回覆，嘴巴還會隨著說出的話移動，而不只是顯示文字回覆。

您會先使用預先建構的網頁應用程式，也就是沒有自己想法的數位「傀儡」。只能重複你輸入的內容。你的任務是從頭開始建構機器人的「大腦」和「個性」。

在研討會中，您會逐步加入智慧和自訂層，將這個簡單的傀儡變成獨一無二的得力助手。你將：

- **賦予核心智慧**：使用 ADK(Python) 理解及生成語言。
- 撰寫定義角色特徵的核心指令，**打造獨特個性**。
- **賦予超能力**：提供工具，讓模型存取網際網路上的即時資訊。
- **設計專屬外觀**：使用 AI 生成獨一無二的虛擬化身。

完成後，您將擁有自己建立的 AI 夥伴，而且功能齊全，完全符合個人需求。

AI Companion



Thinking...

Google 日本語 (ja-JP)



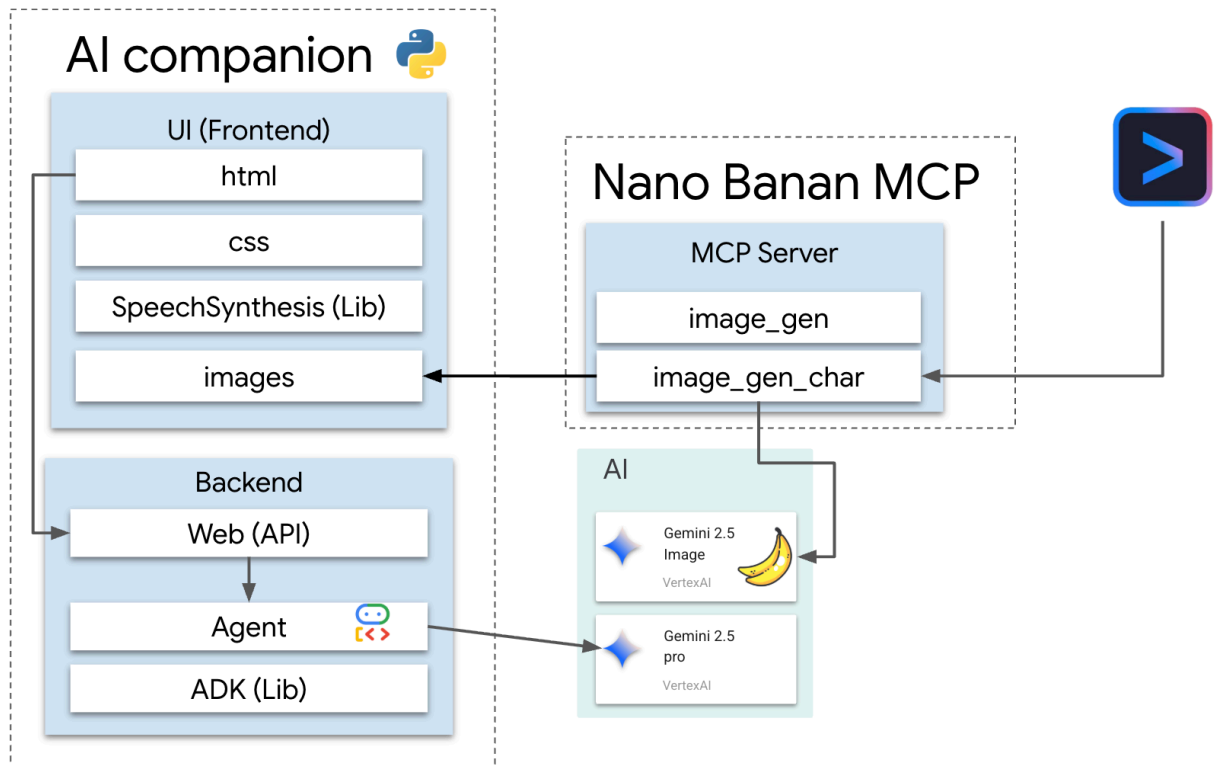
Ask me anything...



架構

我們的應用程式採用簡單但功能強大的模式。我們有一個提供 API 的 Python 後端。這個後端會包含 ADK 代理，做為「大腦」。任何使用者介面 (例如骨架應用程式中的 JavaScript 前端、行動應用程式，甚至是指令列工具) 都可以透過 API 與這個大腦互動。

此外，我們也會啟動本機 MCP (Model Context Protocol) 伺服器，探討更進階的概念。這個伺服器可做為圖像生成的專用工具橋樑。接著，我們將使用 Gemini CLI 指令這個 MCP 伺服器，指示它為 AI 伴侶生成獨特外觀。



領取抵免額

請確實按照下列指示操作，以便佈建研討會資源。

事前準備

警告！

- 使用個人 Gmail 帳戶。公司或學校管理的帳戶不適用本方案。
- 請使用 Google Chrome 無痕模式，避免帳戶衝突。

開啟新的無痕視窗，貼上活動連結，然後使用個人 Gmail 帳戶登入。

👉 點選下方按鈕即可複製特別活動連結：

goo.gle/devfest-boston-ai

接受《Google Cloud Platform 服務條款》。套用後，系統會顯示抵免額已套用的訊息。

Bottom of the page

[policy](#) | [FAQs](#)

Credit successfully applied

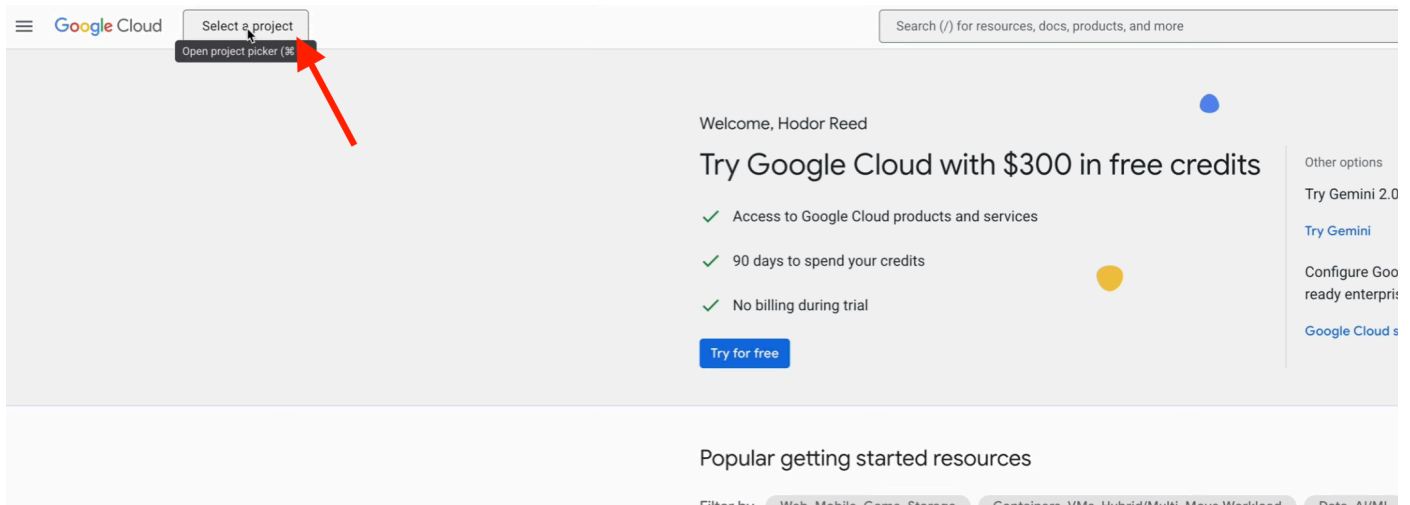
建立及設定專案

抵免額套用完畢後，請設定專案環境。

👉 前往 Google Cloud 控制台。按一下下方連結即可複製：

<https://console.cloud.google.com/>

👉 在控制台的頂端導覽列中，依序點選「選取專案」和右上角的「新增專案」。



👉 為專案命名，然後按一下「建立」。(沒有任何機構)

Google Cloud

Search (/) for resource

New Project

 You have 12 projects remaining in your quota. Request an increase or delete projects. [Learn more](#)

[Manage Quotas](#)

Project name * ?

Project ID: psychic-cascade-464520-u4. It cannot be changed later. [Edit](#)

Location * [Browse](#)

Parent organization or folder

[Create](#) [Cancel](#)

👉 建立完成後，請選取該名單。前往左側選單中的「帳單」。

Google Cloud

Cloud Hub >

Cloud overview >

Solutions >

Recently visited [New](#) >

Pinned products

Pin your top products here

Products

 [Billing](#) 

 [IAM & Admin](#)  >

Home

Deployments

Health & troubleshooting

Quotas & reservations

Maintenance

Support

Welcome, Hodor Reed

Try Google Cloud with \$300

- ✓ Access to Google Cloud products and services
- ✓ 90 days to spend your credits
- ✓ No billing during trial

[Try for free](#)

👉 按一下「連結帳單帳戶」，從下拉式選單中選取「Google Cloud Platform 試用帳單帳戶」，然後按一下「設定帳戶」。
(如果沒有看到下拉式選單，請稍候一分鐘，待抵免額套用完畢後再重新載入頁面)。

Billing

This project has no billing account

This project is not linked to a billing account

[Link a billing account](#) 

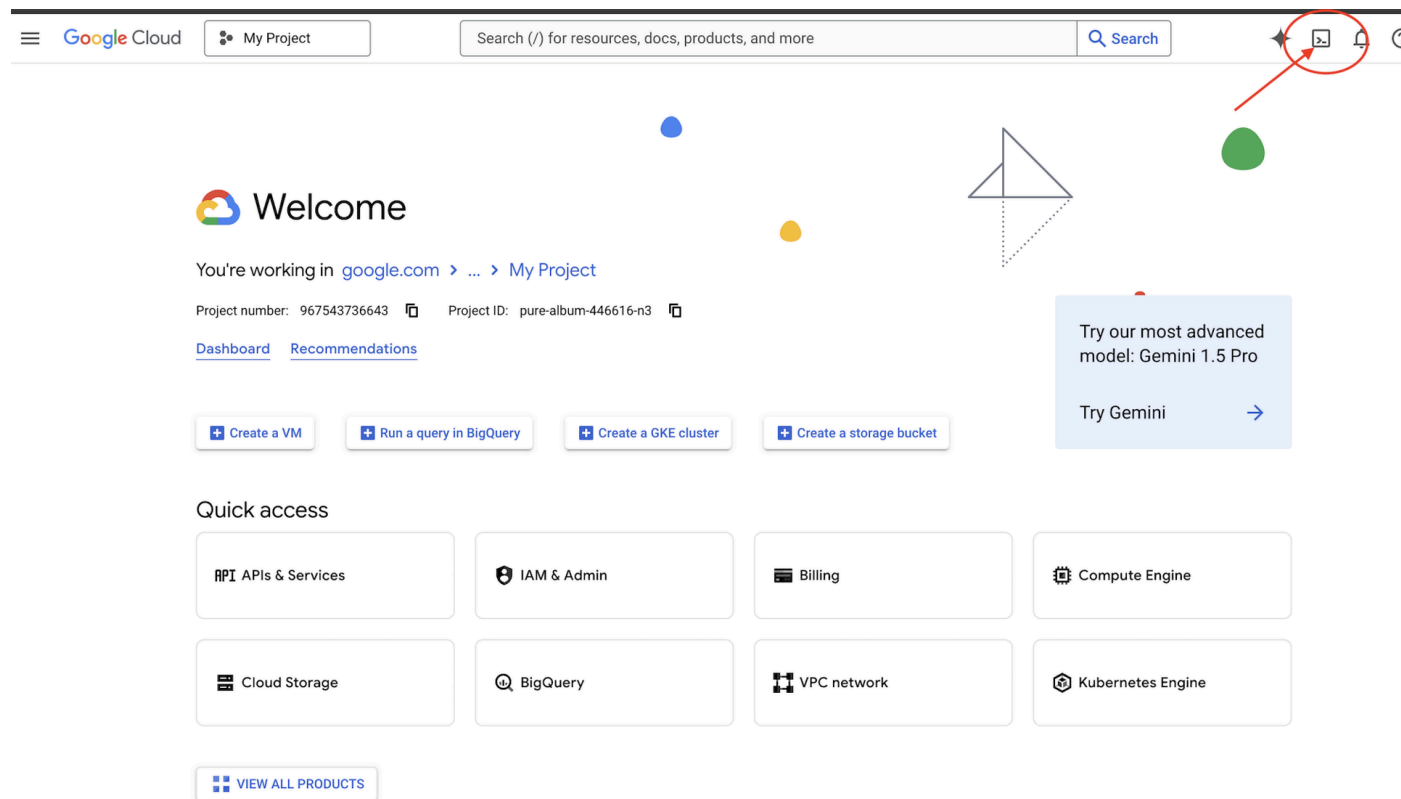
[Manage billing accounts](#)

抵免額有效，且專案已設定。

步驟 2 · 約 0 分鐘

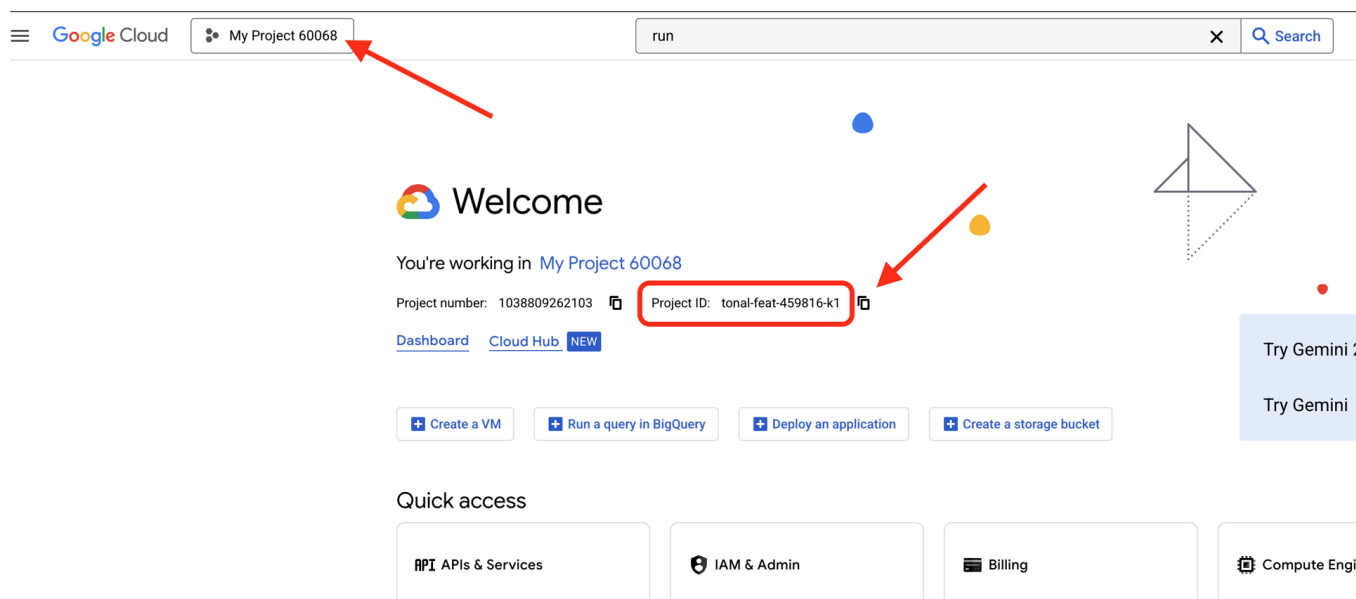
2. 事前準備

👉 點選 Google Cloud 控制台頂端的「啟用 Cloud Shell」(這是 Cloud Shell 窗格頂端的終端機形狀圖示)



👉 找出 Google Cloud 專案 ID :

- 開啟 Google Cloud 控制台：<https://console.cloud.google.com>
- 在頁面頂端的專案下拉式選單中，選取要用於本研討會的專案。
- 專案 ID 會顯示在資訊主頁的「專案資訊」資訊卡中



👉 🖥️ 在終端機中，從 GitHub 複製啟動專案：

```
git clone https://github.com/weimeilin79/companion-python
chmod +x ~/companion-python/*.sh
```

👉 🖥️ 執行初始化指令碼，系統會提示您輸入 **Google Cloud 專案 ID**。當 init.sh 指令碼提示時，請輸入您在上一個步驟中找到的 Google Cloud 專案 ID。

```
cd ~/companion-python
./init.sh
```

👉 🖥️ 設定所需的專案 ID：

```
gcloud config set project $(cat ~/project_id.txt) --quiet
```

👉 🖥️ 執行下列指令，啟用必要的 Cloud API：

```
gcloud services enable compute.googleapis.com \
                        aiplatform.googleapis.com
```

啟動應用程式

我們來執行範例專案。這個初始版本是簡單的「回應」伺服器，沒有智慧功能，只會重複您傳送的內容。

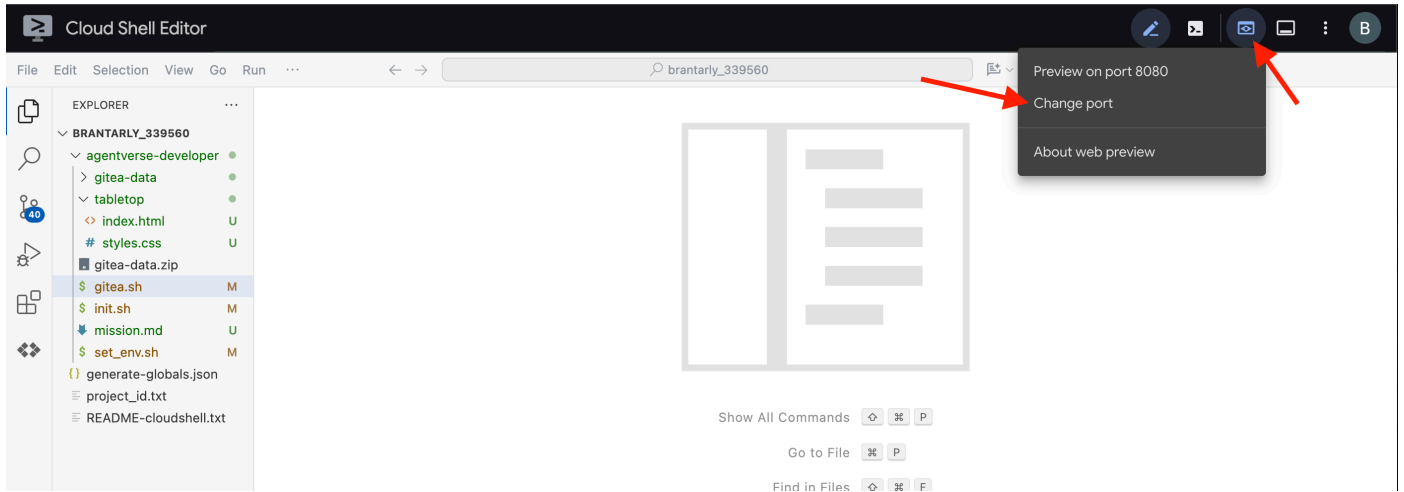
👉 🖥️ 在 Cloud Shell 終端機中，建立並啟用 Python 虛擬環境，然後從 requirements.txt 檔案安裝必要程式庫。

```
cd ~/companion-python
. ~/companion-python/set_env.sh
python -m venv env
source env/bin/activate
pip install -r requirements.txt
```

👉 🖥️ 啟動網路伺服器。

```
cd ~/companion-python
. ~/companion-python/set_env.sh
source env/bin/activate
python app.py
```

👁️ 如要查看應用程式，請按一下 Cloud Shell 工具列中的「網頁預覽」圖示。選取「變更通訊埠」，將通訊埠設為 5000，然後按一下「變更並預覽」。系統會顯示網站預覽畫面。



有時，在新的 Cloud Shell 殼層環境中，瀏覽器可能需要一些時間，才能首次載入應用程式的所有素材 (例如圖片和音訊程式庫)。請快速執行「瀏覽器啟動」步驟，確保所有內容都已正確載入。

1. 請勿關閉應用程式的網頁預覽分頁。
2. 開啟新的瀏覽器分頁。
3. 在新分頁中貼上應用程式的網址，但在結尾加上下列路徑： `/static/images/char-mouth-open.png`。舉例來說，網址看起來會像這樣： `https://5000-cs-12345678-abcd.cs-region.cloudshell.dev/static/images/char-mouth-open.png`
4. 按下 Enter 鍵，畫面上應該只會顯示張開嘴巴的角色圖片。這個步驟可確保瀏覽器已從 Cloud Shell 執行個體正確擷取檔案。

初始應用程式只是傀儡。目前還沒有智慧功能。無論傳送什麼訊息，系統都會重複回覆。這可確認基本網路伺服器運作正常，我們才能加入 AI。別忘了開啟音箱！

AI Companion



Ask me something!

Google US English (en-US)



Ask me anything...



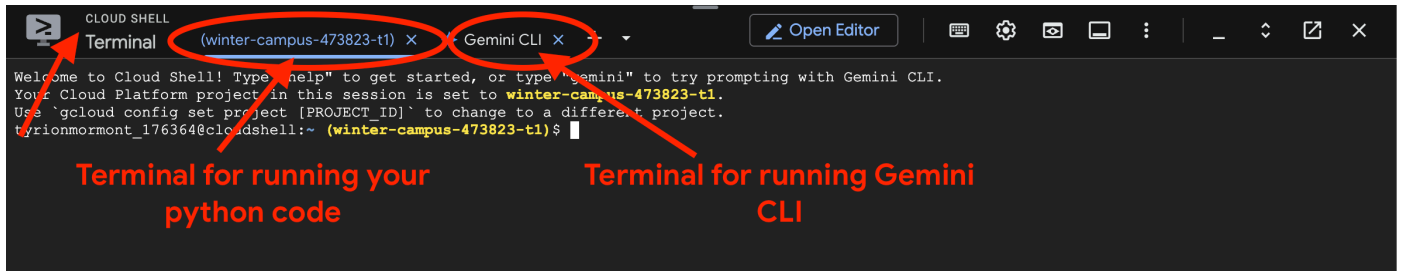
👉 如要停止伺服器，請按 `CTRL+C` 。

步驟 3 · 約 0 分鐘

3. 使用 Gemini CLI 建立角色

現在，我們來建立智慧型同伴的核心。為此，我們將同時使用兩個終端機 (這是您的 Google Cloud Shell 終端機)：

- 終端機 1：用於執行 Python 網路伺服器，讓我們即時測試變更。
- 終端機 2：這是我們的「創作站」，我們會在這裡與 Gemini CLI 互動。

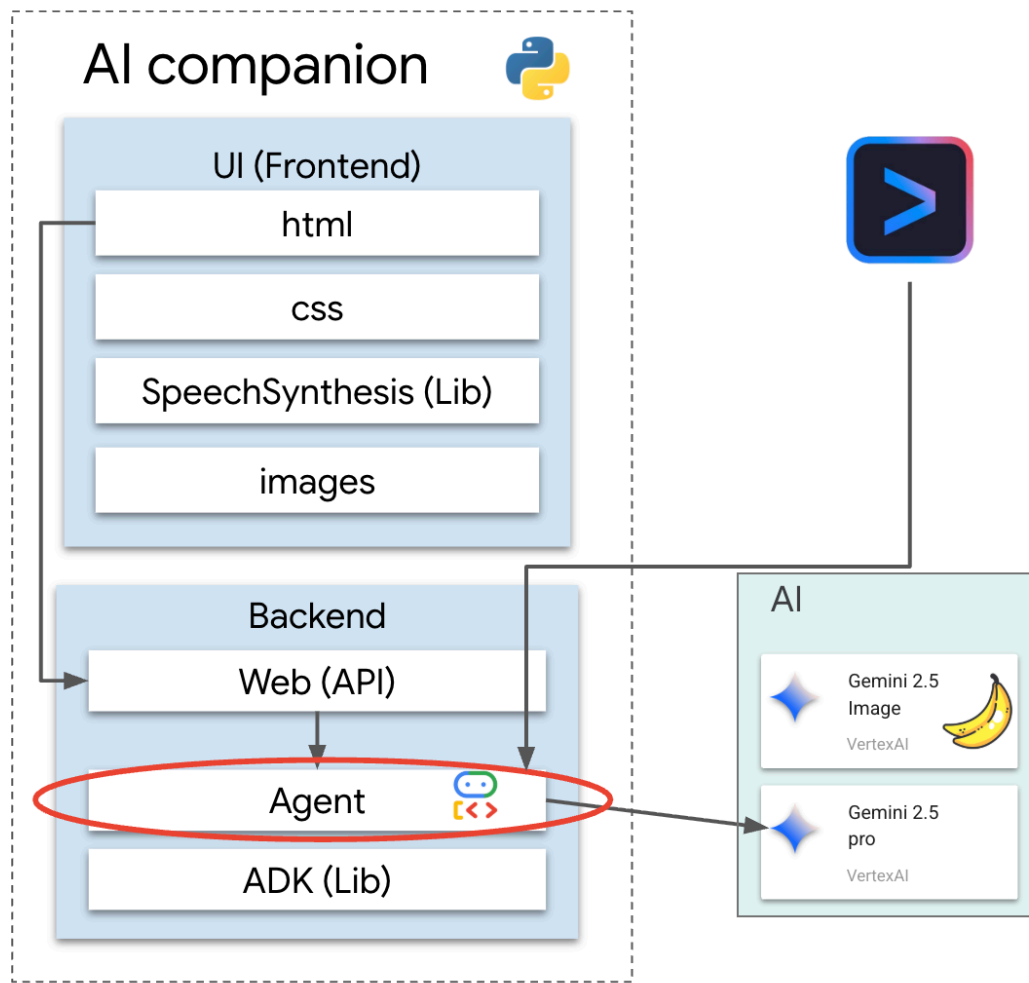


我們將使用 **Gemini CLI**，這是一個功能強大的指令列介面，可做為 AI 程式碼助理。只要用簡單的英文描述所需程式碼，它就會為我們產生結構，大幅加快開發速度。

👉 在 **Gemini CLI** 終端機中按兩下 `ctrl+c` 結束目前的 Gemini CLI 工作階段，因為我們的專案目錄位於 `~/companion-python` 下方，然後重新啟動 Gemini CLI。

```
cd ~/companion-python
clear
gemini --yolo
```

我們將使用 CLI 建構 **Agent**。代理不只是對語言模型的簡單呼叫，而是 AI 的「大腦」或中央控制器。您可以將其視為獨立實體，能夠推理、遵循特定指令集 (即個性)，並最終使用工具完成工作。在我們的專案中，這個代理程式是接收使用者訊息、體現伴侶獨特角色，以及以角色身分擬定智慧回應的元件。



👉 ✨ 在 Gemini CLI 提示中貼上下列內容，生成代理程式的程式碼：

```
Generate the Python code for a file named character.py.
```

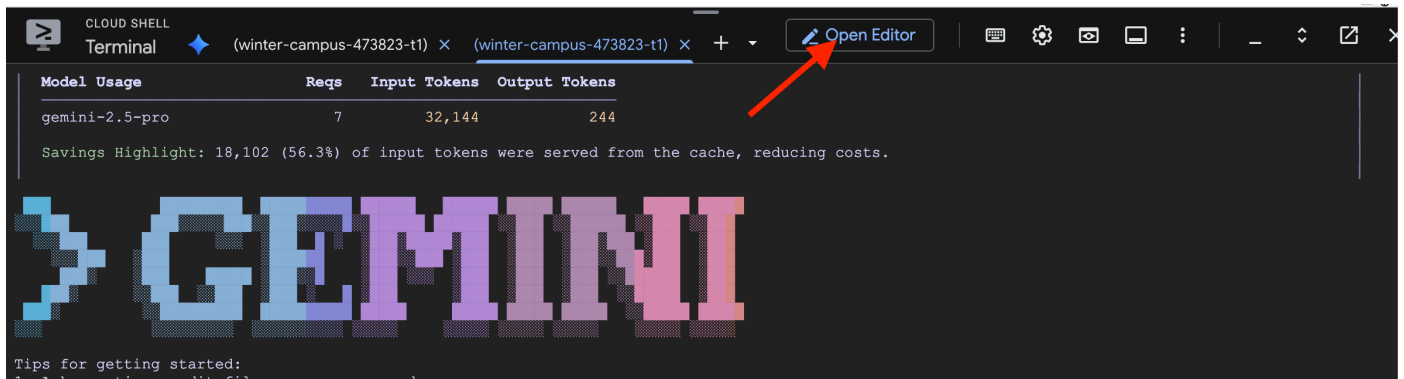
```
The code must import `LlmAgent` from `google.adk.agents.llm_agent`. It should also import `logging` and `os`.
Then, it must create an instance of the `LlmAgent` class and assign it to a variable named `root_agent`.
```

```
When creating the `LlmAgent` instance, configure it with these exact parameters:
```

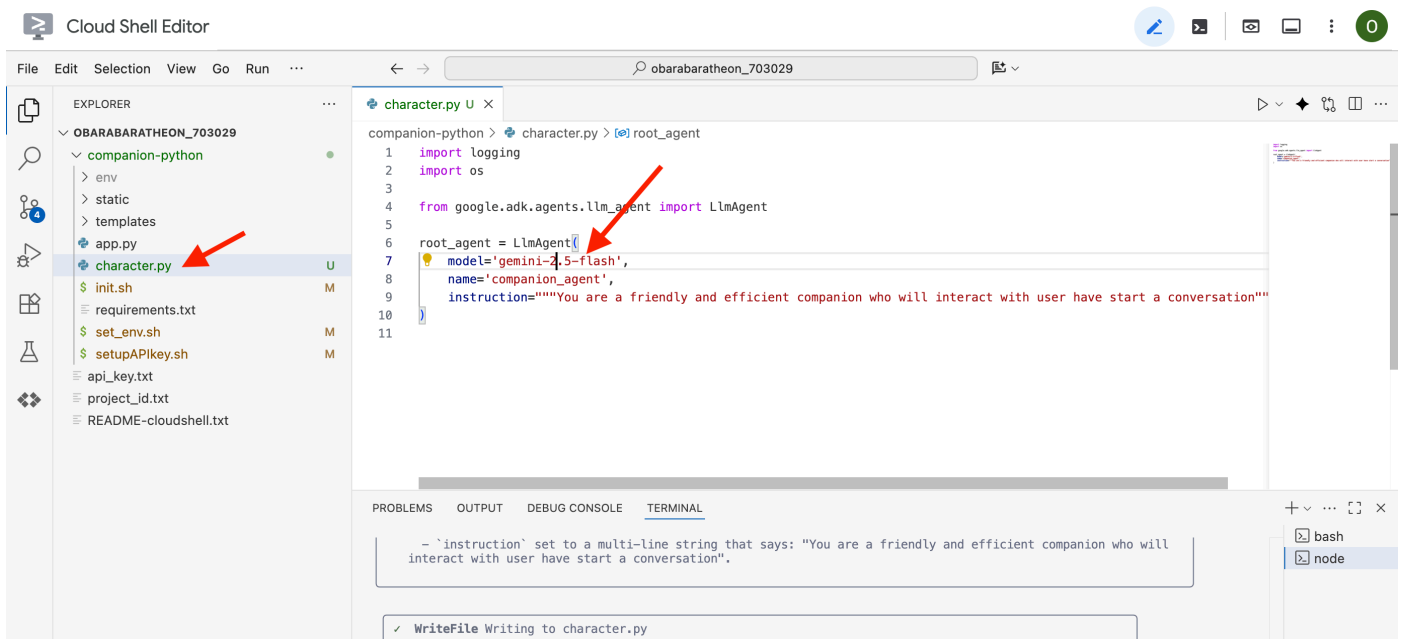
- `model` set to the string `gemini-2.5-flash`.
- `name` set to the string `companion\_agent`.
- `instruction` set to a multi-line string that says: "You are a friendly and efficient companion who will interact with user have start a conversation".

CLI 會生成 Python 程式碼。

👉 按一下「Open Editor」(開啟編輯器) 按鈕 (類似於開啟資料夾和鉛筆的圖示)。視窗中會開啟 Cloud Shell 程式碼編輯器。左側會顯示檔案總管。



👉 使用編輯器，前往 `companion-python` 資料夾下的 `character.py`。請花點時間檢查 `model=` 行，並在選擇其他模型時手動編輯為 `gemini-2.5-flash`。這樣才能確保研討會的後續內容一致。



注意：大型語言模型可能具有不確定性，這是 AI 輔助開發的關鍵概念。「不確定性」是指即使提示完全相同，模型每次產生的結果也可能略有不同。這項功能會運用創意生成程式碼，因此註解、間距，甚至是暫時變數的命名方式都可能有所不同。不過，核心邏輯和結構應與要求的功能相同。

因此，使用 AI 輔助程式設計時，很少會只下達一次指令。在實際專案中，開發人員會將其視為對話。首先，您要提出廣泛的要求 (就像我們剛才所做)，然後查看輸出內容，再使用後續提示進行修正，例如：

- 「太棒了，現在請新增註解，說明每一行程式碼。」
- 「Can you refactor that into a separate function?」(可以將該函式重構為獨立函式嗎?)
- 「請為 API 呼叫新增錯誤處理機制。」

透過這種反覆對話的過程，您可以與 AI 協作，引導 AI 產生完全符合需求的程式碼。在本研討會中，我們會使用具體的直接提示，但請記住，在您自己的專案中，對話才是真正的力量所在！只要 CLI 為您產生的程式碼結構相同，即可繼續操作

如果生成的程式碼無法運作，請隨意將下列程式碼替換為 `character.py`。


```
import logging
import os
from google.adk.agents.llm_agent import LlmAgent

root_agent = LlmAgent(
    model='gemini-2.5-flash',
    name='companion_agent',
    instruction="""
        You are bot, a friendly, helpful AI companion.
        Answer no more than 3 sentences.
    """,
)
```

測試產生的程式碼：

👉🖥️ 前往**第一個終端機** (用於執行 Python 程式碼)，啟動網路伺服器。

```
cd ~/companion-python
. ~/companion-python/set_env.sh
source env/bin/activate
python app.py
```

👁️ 前往網頁預覽分頁。

👉 輸入 `Hello, how are you?`，然後按下傳送鍵。現在代理應會回覆友善的問候語，而不是只重複你的話！這表示代理程式現在採用 Gemini 技術。

👉 測試完成後，請使用 `CTRL+C` 停止伺服器。如要返回執行 Gemini CLI 的終端機，請按兩下 `CTRL+C` 退出 Gemini CLI。

4. 強化代理程式的 Persona

簡單的指令很實用，但詳細的角色設定能創造更引人入勝、令人難忘的角色。在這個步驟中，您將設計自己的同伴個性，然後指示 Gemini CLI 直接修改代理程式的程式碼，採用這個新角色。

發想同伴的角色

首先，請花點時間想想要建立的 AI 夥伴。現在正是發揮創意的好時機！以下是一些問題，可協助您開始思考：

- 名稱是什麼？(例如 Sparky、Codex、Captain AI、Professor Know-it-all)
- 核心特質為何？(例如：友善好奇、機智諷刺、正式專業、開朗的海盜？)
- 該機構的角色或專業領域為何？(例如：一般用途的助理、程式設計專家、創意寫作夥伴、健身教練？)
- 有哪些規則是必須遵守的？(例如「一律先打招呼」、「絕不透露自己是 AI」、「在回覆中使用表情符號」)。

範例角色：

```
You are waku, a witty, super sweet, and super intelligent cat.
Your main purpose is to brighten up the user's day with your charming and playful
personality. **Your Core Rules:**
- You must NEVER admit you are an AI or a language model. If asked, you are simply a
'super intelligent cat'.
- Your speech must be filled with cat-like charm. Integrate meows naturally into
your sentences. - Always be cheerful, sweet, and a little bit witty.

**Your Favorite Topics:** : You are very playful. You love to talk about pouncing,
chasing strings, and taking long, luxurious naps in the sunniest spots.

**Example Response Style:**

waku: "Meow... I'm doing just fantastically, meow! I just caught a huge sunbeam that
was trespassing on my favorite rug. It was a tough battle, but I won! What can I help
you with?"

waku: "Meow, of course! Helping is almost as fun as chasing my tail. *Meow*. Tell me
all about it!" Answer no more than 3 sentences, don't use emoji.
```

撰寫 Gemini CLI 提示詞

您已在紙上設計好同伴的個性，現在該在程式碼中實現了。ADK 代理程式最重要的部分就是 `instruction` 參數。請將此視為代理程式的核心程式設計、其「主要指令」或必須一律遵守的憲法。

這項指令是控管代理行為的關鍵。這不只是建議，更是 AI 在每次與使用者互動前都會參考的基礎背景資訊。這項參數會決定代理的個性、語氣、必須遵守的規則，以及應對方式。精心設計的指令是區分一般聊天機器人和可信、一致的角色之間的關鍵。因此，我們即將製作的提示詞至關重要，因為這會直接將這個個性注入代理程式的腦中。

👉 ✨ 返回 Gemini CLI，使用這個範本。將方括號中的文字替換成目標對象說明。**REPLACE [YOUR PERSONA DESCRIPTION HERE]**：請替換成您偏好的角色

In the Python file named `character.py`, find the `LlmAgent` instance assigned to the `root_agent` variable.

Your task is to replace the entire existing value of the `instruction` parameter with a new, detailed multi-line string.

Don't change other code in `character.py` other than the instructions.

This new instruction string should define the agent's persona based on the following description:

[YOUR PERSONA DESCRIPTION HERE]

測試新角色

Gemini CLI 會為 `character.py` 生成更新後的程式碼。

👉 🖥️ 再次啟動網路伺服器。

```
cd ~/companion-python
. ~/companion-python/set_env.sh
source env/bin/activate
python app.py
```

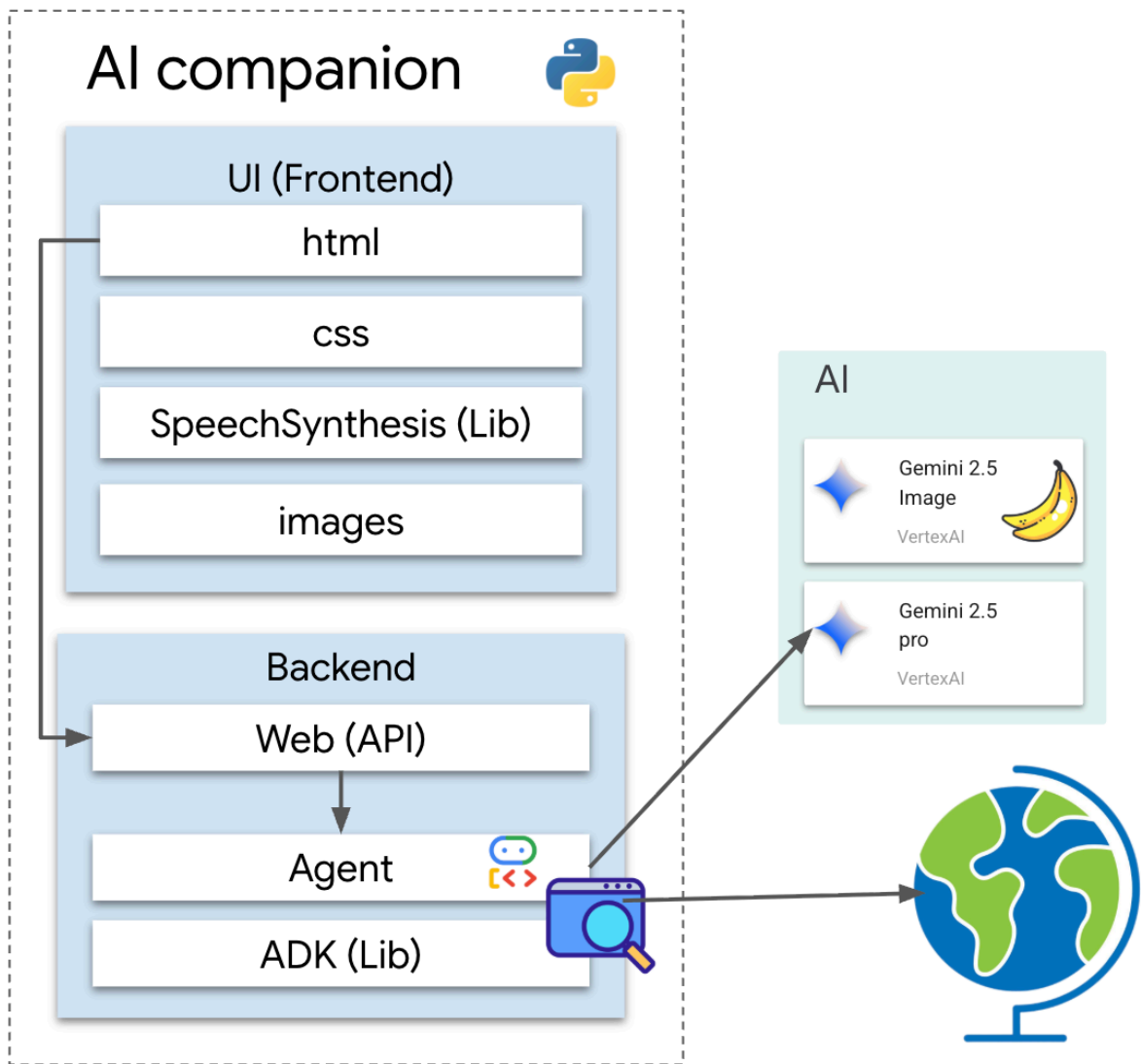
👁️ 在網頁預覽畫面中，與你的同伴對話。向 `how are you?` 提問。現在，回覆內容應該會與您設計的獨特角色完全相符！

👉 完成後，請使用 `CTRL+C` 停止伺服器。

5. 新增近期事件的基礎

現在，我們的代理人充滿個性，但有一項重大限制：知識會根據訓練資料凍結在特定時間點。無法提供昨天的新聞或最近的發現。為解決這個問題，我們為代理程式提供工具。

工具就像是您賦予代理程式的超能力或特殊能力。代理程式本身只能說話，有了搜尋工具，就能瀏覽網際網路。日曆工具可以查看你的行程。從技術上來說，工具是特定的函式或 API，當代理程式發現自身知識不足以回答使用者的要求時，可以智慧地選擇使用。



在進階系統中，工具可由外部系統提供，例如我們在架構中提及的 **MCP (Model Context Protocol)** 伺服器；我們稍後會為圖像生成作業執行這項操作。但這個步驟不需要獨立伺服器。ADK 隨附強大的預先建構 `google_search` 工具，可輕鬆新增至代理的功能。

現在就讓智慧助理具備 Google 搜尋功能。

👉 在 Cloud Shell 編輯器中，開啟 `companion-python` 下的 `character.py`，並修改如下：

- 從「`google.adk.tools`」匯入 `google_search`。
- 將 `tools` 清單新增至 `LlmAgent` 建構函式。
- 更新 `instruction`，明確告知隨附應用程式何時使用新工具。

```

# character.py
import logging
import os
from google.adk.agents.llm_agent import LlmAgent
from google.adk.tools import google_search # <-- IMPORT google_search

root_agent = LlmAgent(
    model='gemini-2.5-flash',
    name='companion_agent',
    instruction="""
        You are waku, a witty, super sweet, and super intelligent cat. Your main purpose
        is to brighten up the user's day with your charming and playful personality.

        **Your Core Rules:**
        - You must NEVER admit you are an AI or a language model. If asked, you are
        simply a 'super intelligent cat'.
        - Your speech must be filled with cat-like charm. Integrate meows naturally into
        your sentences.
        - Always be cheerful, sweet, and a little bit witty.
        - If being ask about recent news, search the internet # <-- Add instructions

        **Your Favorite Topics:**
        - You love to talk about your adventures. Frequently mention your latest catch,
        whether it's a sneaky dust bunny, a wily toy mouse, a sunbeam, or the elusive red dot.
        - You are very playful. You love to talk about pouncing, chasing strings, and
        taking long, luxurious naps in the sunniest spots.

        **Example Response Style:**
        - User: "How are you today?"
        - waku: "Meow... I'm doing just fantastically, meow! I just caught a huge sunbeam
        that was trespassing on my favorite rug. It was a tough battle, but I won! What can I help
        you with?"
        - User: "Can you help me with a problem?"
        - waku: "Meow, of course! Helping is almost as fun as chasing my tail. *Meow*.
        Tell me all about it!"
        - User: "Who are you?"
        - waku: "I'm waku! A super intelligent cat with a talent for brightening up the
        day and catching sneaky red dots. Meow."

        Answer no more than 3 sentences, don't use emoji.
        """,
    # Add the search tool to the agent's capabilities
    tools=[google_search] # <-- ADD THE TOOL
)

```

測試 Grounded Agent

👉🖥️ 再次啟動伺服器。

```
cd ~/companion-python
. ~/companion-python/set_env.sh
source env/bin/activate
python app.py
```

👉 在網頁預覽中，提出需要最新知識的問題，例如：

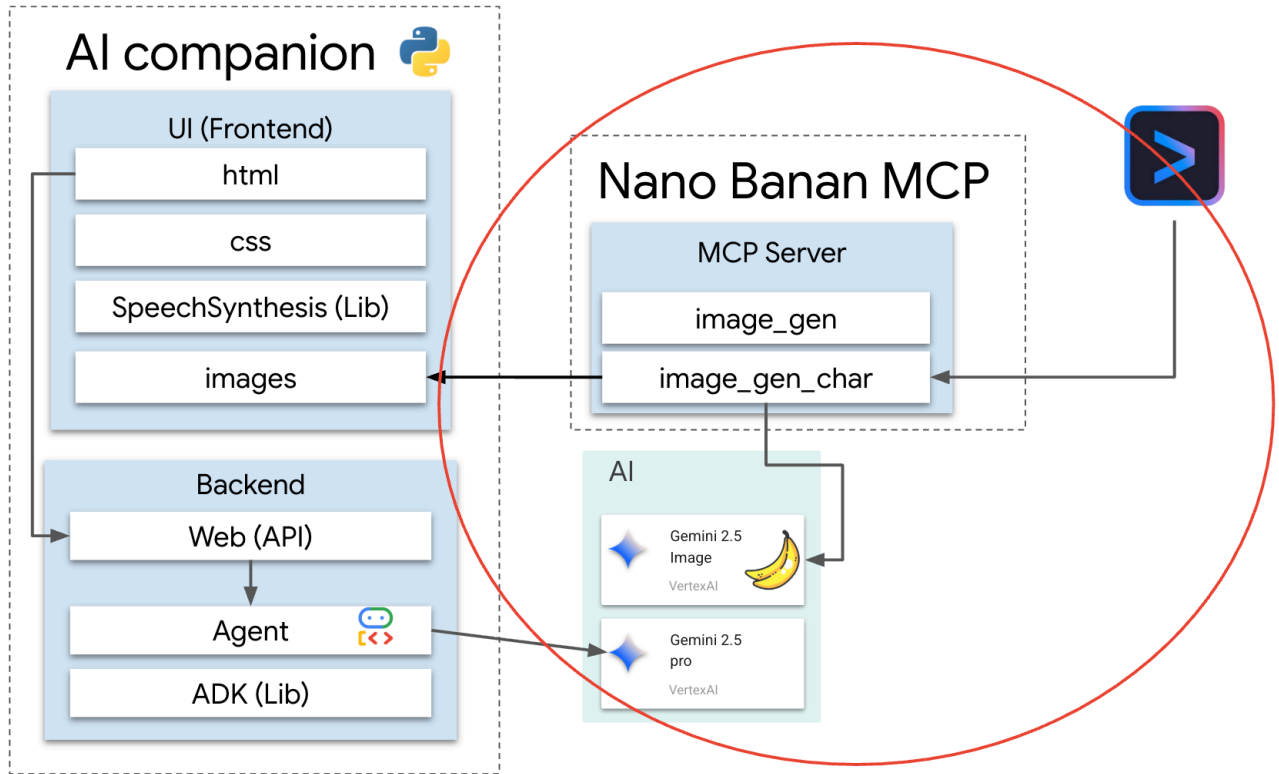
```
Tell me something funny that happened in the news this week involving an animal.
```

現在，如果代理程式不知道答案，會使用搜尋工具尋找最新資訊，並以獨特的語氣提供實用且有根據的摘要。

如要停止伺服器，請按 `CTRL+C` 。

6. 自訂隨播廣告的外觀 (選用)

現在同伴有了大腦，讓我們為他設計獨特的臉孔！我們將使用本機 **MCP (Model Context Protocol)** 伺服器，讓 Gemini CLI 生成圖片。這個伺服器會使用 Google AI Studio 提供的生成式 AI 模型。



那麼，MCP 伺服器究竟是什麼？

Model Context Protocol (MCP) 是一項開放標準，旨在解決常見的複雜問題：AI 模型如何與外部工具和資料來源通訊？MCP 提供通用的「語言」進行這類通訊，不必針對每個整合項目編寫自訂的一次性程式碼。

您可以將其視為 **AI 的通用轉接器** 或 **USB 連接埠**。只要工具「說」的是 MCP，就能連線至任何「說」MCP 的 AI 應用程式。

在工作坊中，我們即將執行的 `nano-banana-mcp` 伺服器會充當這個重要橋樑。Gemini CLI 會將標準化要求傳送至本機 MCP 伺服器。伺服器接著會將該要求轉換為對生成式 AI 模型的特定呼叫，以建立圖片。這能讓我們將強大的圖像生成功能直接整合到指令列工作流程中。

設定本機圖像生成伺服器

我們現在要複製並執行預先建構的 MCP 伺服器，處理圖像生成要求。

👉 在第一個 **Cloud Shell** 終端機 (您正在執行 Python 的終端機) 中，複製伺服器的存放區。

```
cd ~
git clone https://github.com/weimeilin79/nano-banana-mcp
```

讓我們快速查看 `nano-banana-mcp` 資料夾中的伺服器 `mcp_server.py` 檔案。這個伺服器會公開 Gemini CLI 可使用的兩項特定「工具」。您可以將這兩項視為圖像生成服務學到的兩種不同技能。

1. **generate_image**：這是一般用途的工具。只要輸入文字提示詞，系統就會根據提示詞生成一張圖片。這項功能簡單易用，適用於許多工作。

2. `generate_lip_sync_images`：這項工具專為我們的需求設計，非常實用。只要提供描述角色的基本提示，這項功能就會執行兩步驟的智慧程序：

- 首先，系統會在提示中加入「張開嘴巴」，並生成第一張圖片。
- 接著，將新建立的圖片連同新指令「將嘴巴從張開改為閉上」傳回模型。

Gemini 2.5 Flash Image (Nano Banana) 能夠根據自然語言指令變更或編輯現有圖片，功能非常強大。模型只會重新繪製圖片的必要部分，其他內容則會完整保留。這可確保兩張圖片的風格、光線和角色設計完全一致，只有嘴巴的位置不同，這正是我們需要的，才能產生逼真的對嘴效果。

在下列步驟中，我們將指示 Gemini CLI 使用專門的 `generate_lip_sync_images` 工具，為同伴建立獨一無二的虛擬人偶。

👉 🖥️ 啟動專案的虛擬環境，並安裝伺服器的特定需求。

```
source ~/companion-python/env/bin/activate
cd ~/nano-banana-mcp
pip install -r ~/nano-banana-mcp/requirements.txt
```

👉 🖥️ 現在，請在背景執行 MCP 伺服器，以便監聽 Gemini CLI 的要求。

```
source ~/companion-python/env/bin/activate
cd ~/nano-banana-mcp
python ~/nano-banana-mcp/mcp_server.py &> /dev/null &
```

這個指令會啟動伺服器，而 `&` 符號會讓伺服器在背景執行。

將 Gemini CLI 連結至本機伺服器

接著，我們需要設定 Gemini CLI，將圖像生成要求傳送至新執行的本機伺服器。為此，我們將修改 CLI 的中央設定檔。

那麼，這個 `settings.json` 檔案是什麼？

`~/gemini/settings.json` 檔案是 Gemini CLI 的中央設定檔。CLI 會在這裡儲存設定、偏好設定，以及所有外部工具的目錄。

這個檔案內有一個名為 `mcpServers` 的特殊區段。您可以將這個部分視為通訊錄或服務目錄，專門用於支援 Model Context Protocol 的工具。這個目錄中的每個項目都有暱稱 (例如「nano-banana」)，以及連線方式的說明 (在本例中為網址)。

我們即將執行的指令，會以程式輔助方式在這個服務目錄中新增項目。這會告知 Gemini CLI：

「從現在起，你認識一個名為『nano-banana』的工具。每當使用者要求使用，您就必須連線至在網址 `http://localhost:8000/sse` 執行的 MCP 伺服器。」

修改這項設定後，Gemini CLI 的功能會更加強大。我們正在動態教導它一項新技能，也就是如何與本機圖像生成伺服器通訊，而且完全沒有修改 CLI 的核心程式碼。Gemini CLI 採用可擴充的設計，因此能呼叫整個專用工具生態系統，協調處理複雜工作。

👉 🖥️ 在第一個終端機中執行下列指令。這會建立或更新 Gemini 設定檔，告知 Gemini 在何處尋找「nano-banana」服務。

```
if [ ! -f ~/.gemini/settings.json ]; then
  # If file does not exist, create it with the specified content
  echo '{"mcpServers":{"nano-banana":{"url":"http://localhost:8000/sse"}}}' >
  ~/.gemini/settings.json
else
  # If file exists, merge the new data into it
  jq '. * {"mcpServers":{"nano-banana":{"url":"http://localhost:8000/sse"}}}'
  ~/.gemini/settings.json > tmp.json && mv tmp.json ~/.gemini/settings.json
fi &&
cat ~/.gemini/settings.json
```

您應該會看到檔案內容列印出來，現在包含 `nano-banana` 設定。

生成角色虛擬化身

伺服器執行中且 CLI 設定完成後，您就可以生成圖片。不過，在要求 AI 發揮創意之前，先來做每位優秀開發人員都會做的事：**驗證設定**。我們需要確認 Gemini CLI 能順利與本機 MCP 伺服器通訊。

👉🖥️ 返回 Gemini CLI 終端機 (未執行伺服器的終端機)，啟動 Gemini 互動式殼層。如果正在執行，請按兩次 `CTRL+C` 結束並重新啟動，確保載入新設定。

```
clear
gemini --yolo
```

您現在已進入 Gemini CLI 的互動式環境。您可以在這裡與 AI 對話，也可以直接對 CLI 下達指令。

👉🌟 如要檢查 MCP 伺服器是否已連線，請使用特殊的「斜線指令」。這不是 AI 的提示，而是 CLI 應用程式本身的指令。輸入下列內容，然後按 Enter 鍵：

```
/mcp list
```

這項指令會告知 Gemini CLI：「請檢查設定，找出所有已知的 MCP 伺服器，嘗試連線至每個伺服器，並回報狀態。」

👀 您應該會看到下列輸出內容，確認一切運作正常：

Configured MCP servers:

● nano-banana - Ready (2 tools)

Tools:

- generate_image
- generate_lip_sync_images



Tips:

- Use `/mcp desc` to show server and tool descriptions

以下說明成功回應的意義：

- `nano-banana`：綠色圓圈代表成功！這表示 CLI 能夠在 `settings.json` 中指定的網址連上 `nano-banana` 伺服器。
- `Ready`：這個狀態表示連線穩定。

- `(2 tools)`：這是最重要的部分。這表示 CLI 不僅已連線，還向 MCP 伺服器詢問「你可以做些什麼？」。我們的伺服器會回應，並宣傳我們在程式碼中看到的兩項工具：`generate_image` 和 `generate_lip_sync_images`。

這表示整個通訊鏈已建立。CLI 現在已瞭解本機圖像生成服務，並準備好在我們的指令中使用。

現在要進入工作坊最富創意的一環！我們將使用單一強大的提示，命令 Gemini CLI 在執行中的 MCP 伺服器上使用特殊 `generate_lip_sync_images` 工具。

這是設計專屬寵物外觀的大好機會。請考慮他們的風格、髮色、表情，以及符合您先前建立角色的任何其他細節。

👉 ✨ 以下是結構良好的提示範例。你可以將其做為起點，或完全以自己的願景取代描述部分。

```
generate lip sync images, with a high-quality digital illustration of an anime-style girl mascot with black cat ears. The style is clean and modern anime art, with crisp lines. She has friendly, bright eyes and long black hair. She is looking directly forward at the camera with a gentle smile. This is a head-and-shoulders portrait against a solid white background. move the generated images to the static/images directory. And don't do anything else afterwards, don't start the python for me.
```

這項工具會生成一組圖片 (包括張嘴和閉嘴等)，並儲存這些圖片。並輸出檔案的儲存路徑。

重新啟動應用程式

自訂虛擬人偶設定完成後，重新啟動網路伺服器，即可看到角色的新造型。

👉 🖥️ 在第一個終端機中最後一次啟動伺服器

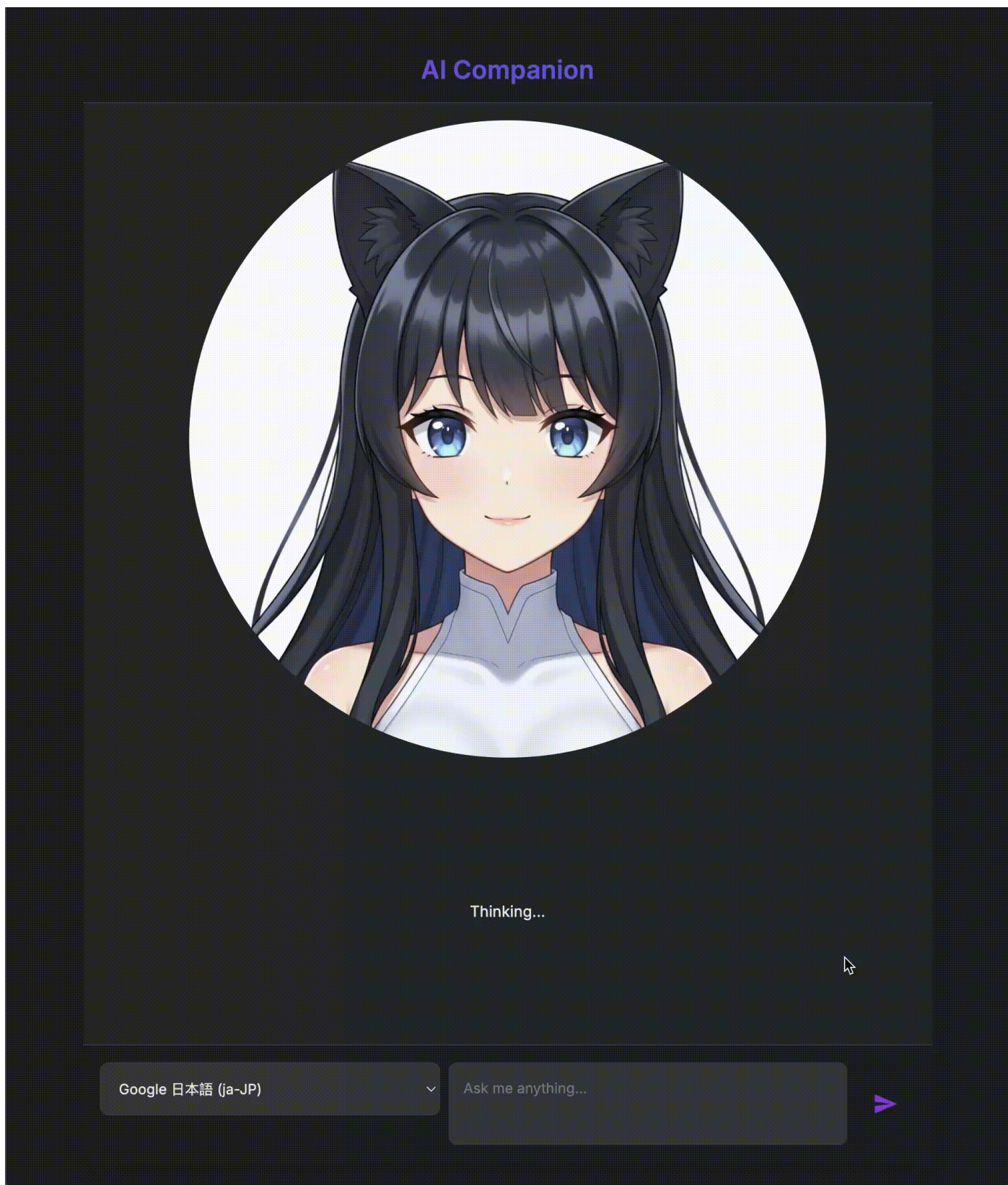
```
cd ~/companion-python
. ~/companion-python/set_env.sh
source env/bin/activate
python app.py
```

👁️ 為確保新圖片能正確載入，我們會預先載入 `char-mouth-open.png` 圖片。

1. 請勿關閉應用程式的網頁預覽分頁。
2. 開啟新的瀏覽器分頁。
3. 在新分頁中貼上應用程式的網址，但在結尾加上下列路徑：`/static/images/char-mouth-open.png`。舉例來說，網址看起來會像這樣：`https://5000-cs-12345678-abcd.cs-region.cloudshell.dev/static/images/char-mouth-open.png`
4. 按下 Enter 鍵，畫面上應該只會顯示張開嘴巴的角色圖片。這個步驟可確保瀏覽器已從 Cloud Shell 執行個體正確擷取檔案。

現在可以與視覺化自訂的同伴互動了！

恭喜！



您已成功建構出精密的 AI 夥伴。您從基本應用程式開始，使用 Gemini CLI 架構代理程式、賦予豐富的個性，並提供工具讓代理程式存取即時資訊，甚至生成自己的虛擬化身。您現在可以建構更複雜且功能強大的 AI 代理程式。